BAX-423 Big Data · Spring 2026 · Yang, Alice · UC Davis GSM

Problem Statement

Developers and founders waste hours manually scanning GitHub and Hacker News to find where to contribute, comment, or build reputation. EngageIQ automates this by ingesting 10,500+ opportunities across 15 technical domains, scoring them via semantic embeddings and community signals, and adapting rankings in real time from user feedback.

System Architecture

Stage	Component	BAX-423 Lecture
1 · Ingestion	GitHub API · HN Firebase API Bloom Filter dedup · Python queue-based on-demand refresh	Lecture 2
2 · Embedding	Sentence-BERT all-MiniLM-L6-v2 (384-dim) FAISS IndexFlatIP — exact search < 2 ms	Lecture 5
3 · Scoring	Composite: 0.40 relevance + 0.30 community + 0.20 visibility + 0.10 (1–effort)	Lecture 7
4 · Ranking	Multi-stage: candidate gen → composite → bandit Diversity re-rank (max 5/domain in top-50)	Lecture 7
5 · Adaptation	Thompson Sampling (Beta-Bernoulli bandit) Domain preference learning over 50+ rounds	Lecture 8
6 · Analytics	Pandas batch: domain health, trending, volume	Lecture 3
7 · Dashboard	Streamlit: ranked cards, Why this?, CSV/JSON brief	—

Dataset

Metric	Value
Total records	10,500
Technical domains	15 (700 records each)
Sources	GitHub 7,500 · Hacker News 3,000
Storage	CSV offline snapshot (committed to repo) + SQLite for live records
Live / offline split	10,500 offline snapshot · live records added via on-demand refresh

BAX-423 Techniques & Benchmarks

Integrated BAX-423 Techniques

Technique	Lecture	File	Role	Benchmark
Bloom Filter (sketching/dedup)	Lec 2	bloom_filter.py	Rejects duplicate streamed records before they enter the ranking pool	0% collision rate on 10,500 unique IDs
Sentence-BERT + FAISS IndexFlatIP	Lec 5	embeddings.py	384-dim semantic embeddings; exact inner-product search < 2 ms per query at 10k scale	Query latency: <2 ms Embedding NDCG@10: 1.00
Multi-stage Ranking (NDCG@10)	Lec 7	ranking.py	4-component composite score + diversity re-rank; NDCG@10 outperforms stars-only baseline	NDCG@10: 0.622 vs stars-only: 0.454
Thompson Sampling (RL bandit)	Lec 8	adaptive_learning.py	Beta-Bernoulli posterior per opportunity; domain preference score learns from feedback	50 rounds: ML/AI pref score 0.50→1.0

Ranking Benchmark — NDCG@10 (Sofia Persona, n=500)

Relevance defined as: 1.0 if domain in {Machine Learning, AI Research}, else 0.1. Sample: first 500 records. Random seed fixed at 42.

Ranking Method	NDCG@10	Notes
Random Baseline	0.4438	Shuffle — no signal
Stars-Only Ranking	0.4544	Popularity ≠ persona relevance
Embedding Similarity Only	1.0000	Semantic query match, no diversity
Full Composite + Re-rank ✓	**0.6221**	**Balances relevance + community + diversity**

Adaptive Learning — Thompson Sampling (50 Rounds)

Synthetic Sofia persona: engage if domain ∈ {ML, AI Research}, skip otherwise. Thompson Sampling Beta posterior updated each round. Random seed = 42.

Metric	Before (Round 0)	After (Round 50)
ML + AI Research domain pref score	0.50 (uniform prior)	1.00 (maximum)
Other 13 domains avg pref score	0.50 (uniform prior)	0.00 (minimized)
Preferred / non-preferred score gap	0.00	+1.00

6 Core Capabilities & Persona Test Results

6 Core Capabilities

#	Capability	Implementation
	Multi-Source Ingestion & On-Demand Refresh	GitHub API · Hacker News Firebase API Bloom Filter dedup · Python queue-based live refresh
2	**Content Embedding & Similarity Retrieval**	all-MiniLM-L6-v2 (384-dim) · FAISS IndexFlatIP Embedding cache for <2 ms queries
	Engagement Scoring & Multi-Stage Ranking	4-component composite + diversity re-rank NDCG@10 = 0.622 vs 0.444 random baseline
4	**Adaptive Learning from Feedback**	Thompson Sampling Beta-Bernoulli bandit Domain pref 0.50→1.00 over 50 rounds
	Batch Analytics & Trend Detection	Pandas batch: domain health, trending repos Volume-over-time, rising opportunities
6	**Dashboard & Engagement Brief**	Streamlit: ranked cards, Why this?, suggested actions CSV/JSON engagement brief export

Persona Pass/Fail Test Results

Each persona was tested against all 6 capabilities. Pass = feature surfaces expected results for that persona's interests.

Persona / Role	1 Ingest	2 Embed	3 Rank	4 Adapt	5 Analytics	6 Brief	Result
Sofia ML Student / Portfolio	✓ Pass	✓ Pass	✓ Pass	✓ Pass	✓ Pass	✓ Pass	**PASS**
David DevOps / Niche Community	✓ Pass	✓ Pass	✓ Pass	✓ Pass	✓ Pass	✓ Pass	**PASS**
Lina Data Journalist / Trends	✓ Pass	✓ Pass	✓ Pass	✓ Pass	✓ Pass	✓ Pass	**PASS**
Raj Startup Founder / B2B	✓ Pass	✓ Pass	✓ Pass	✓ Pass	✓ Pass	✓ Pass	**PASS**

Evidence notes: Sofia top-10 results surface ML Research and AI Research repos with semantic similarity ≥ 0.85. David's results lead with DevOps/K8s and Cloud API repos. Lina's results include trending open-source across all domains sorted by growth_rate. Raj's results highlight Developer Tools and B2B SaaS repos by community engagement.

Limitations & Future Work

Honest Scope — Course-Project Implementation

Area	Current Implementation	Production Equivalent
Streaming	On-demand API refresh + Python queue/thread simulation	Apache Kafka / Flink persistent streaming cluster
Data Sources	GitHub + Hacker News (two sources meets spec requirement; Reddit excluded — OAuth2 API access unavailable in this scope)	Expand to authenticated APIs: Reddit PRAW, LinkedIn, DEV.to
AI Suggestions	Deterministic template-based engagement action generator (avoids API cost, latency, hallucination risk)	LLM-generated actions via Claude / GPT with prompt caching
Batch Analytics	Pandas batch over 10,500-record offline snapshot (sufficient for dataset size, <1 s query latency)	Apache Spark / Dask for distributed processing at scale
GH Archive	build_real_dataset.py includes GH Archive support; not used in runtime pipeline	Scheduled GH Archive pulls as supplemental source

Deployment Notes

App requires **sentence-transformers** and **faiss-cpu** (≈450 MB combined). On first launch, embeddings are computed once and cached to `data/embeddings.npy` (~16 MB). Subsequent launches load the cache in <1 second. Streamlit Community Cloud deployment may require a free-tier instance with ≥1 GB RAM. Live API fetch is *optional* — the app runs fully on the pre-seeded offline dataset.

Future Work

• Connect suggested engagement actions to a live LLM (e.g., Claude claude-haiku-4-5-20251001) for natural-language engagement drafts; cache generated briefs to minimize API cost.

• Expand data sources to Reddit (authenticated PRAW), LinkedIn, and DEV.to for broader community coverage.

• Add scheduled GH Archive pulls for historical trend analysis beyond real-time API limits.

• Extend Thompson Sampling to opportunity-type preferences (issue vs. repo vs. post) for finer-grained adaptive ranking.

• Replace Pandas batch analytics with Dask or Spark when dataset grows beyond 100k records.

Submission Checklist

Item	Status
code/ — all .py files + requirements.txt	✓ **Included**
data/opportunities.csv — 10,500 offline records (GitHub + HN)	✓ **Included**
data/embeddings.npy — pre-computed 384-dim embeddings	✓ **Included**
brief.pdf — this document	✓ **Included**
prompts.md — development + planned runtime prompts	✓ **Included**
Live public URL	**https://engageiq-bax423git-qianyingyang.streaml**
ZIP: Yang_Alice_BAX423_Final.zip	✓ Per one-pager spec